

Gluon

Tile-Based GPU Programming with Low-Level Control

Peter Bell, Mario Lezcano, Keren Zhou
OpenAI



Agenda

- **Introduction to gluon**
 - Motivation & comparison to Triton
 - Language design
- **Linear Layouts Deep-Dive**
- **Case Study: Optimized Blackwell Matmul**
- **Developer tools**
 - Profiler
 - Sanitizers
 - Layout visualizer



Triton

```
@triton.jit
def matmul_kernel(A, B, C):
    # Compute C = A @ B
    m, n = compute_idxes(A, B, C)
    acc = tl.zeros(C.block_shape, dtype=tl.float32)
    for k in tl.range(0, A.shape[1], A.block_shape[1],
                      num_stages=4, warp_specialize=True):
        a = A.load([m, k])    # [BLOCK_M, BLOCK_K]
        b = B.load([n, k]).T  # [BLOCK_K, BLOCK_N]
        acc += tl.dot(a, b)   # [BLOCK_M, BLOCK_N]
    C.store([m, n], acc)
```

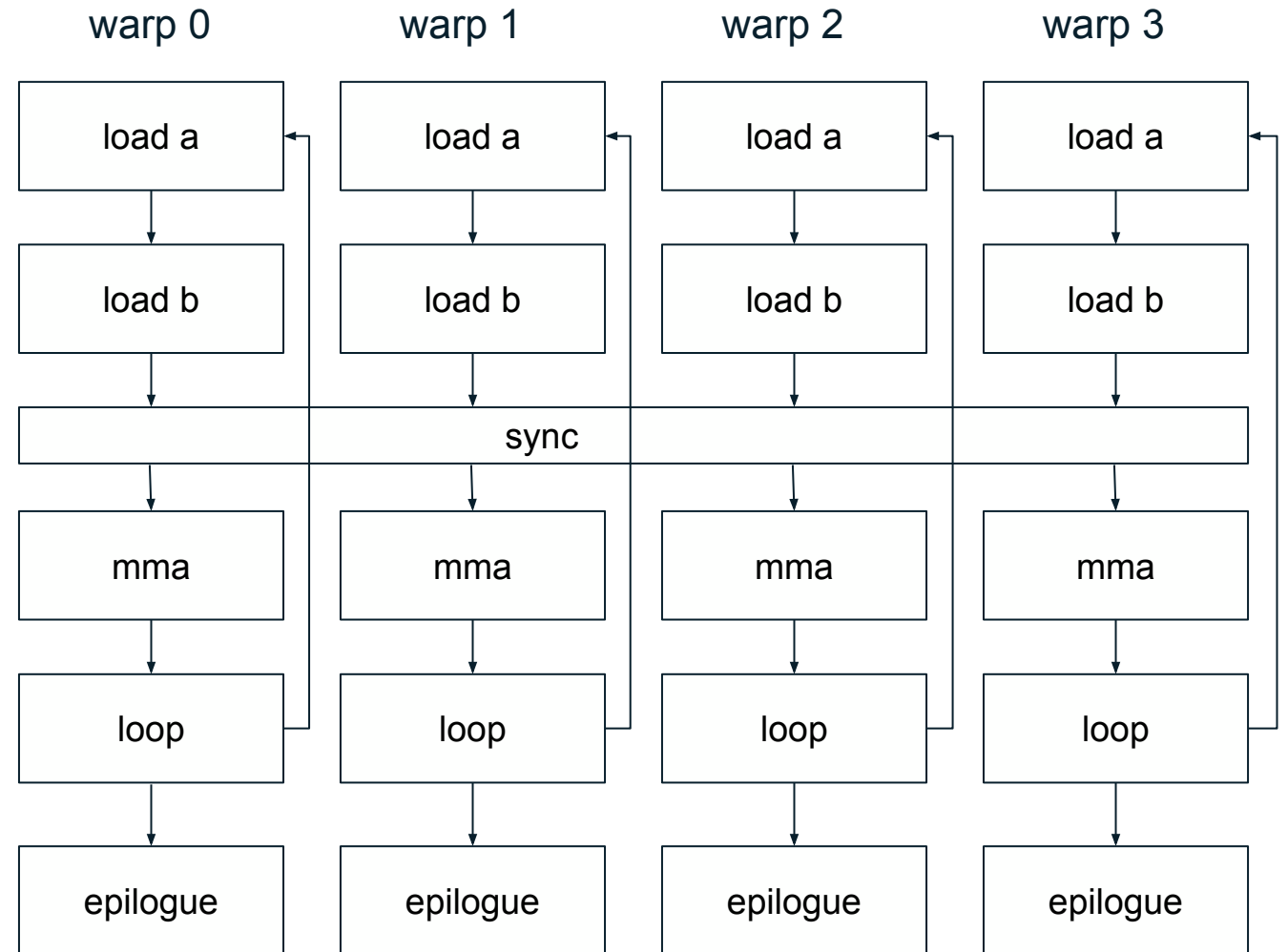
- High level, tile-based GPU programming
- Good performance, excellent effort/reward ratio
- Compiler handles a lot of complexity for the programmer



Why we need a new language

Ampere

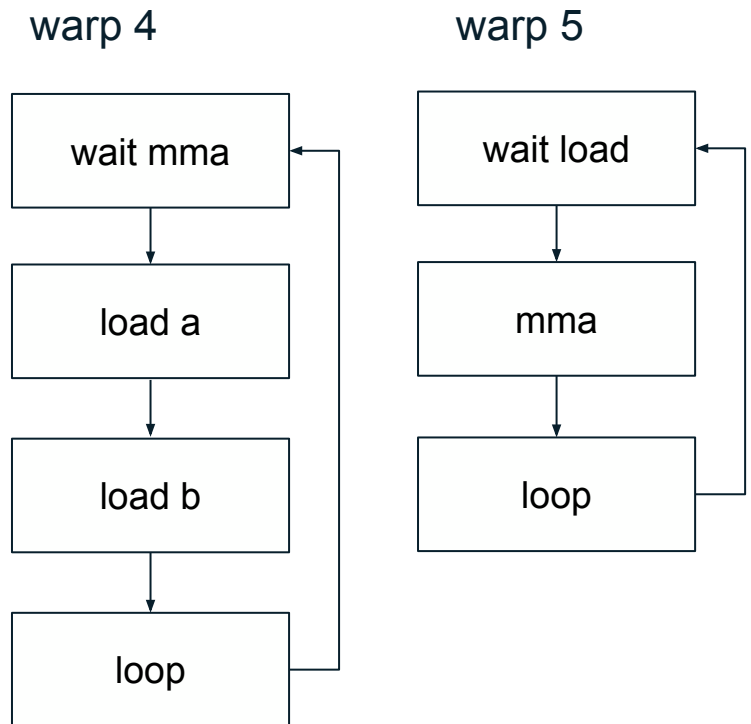
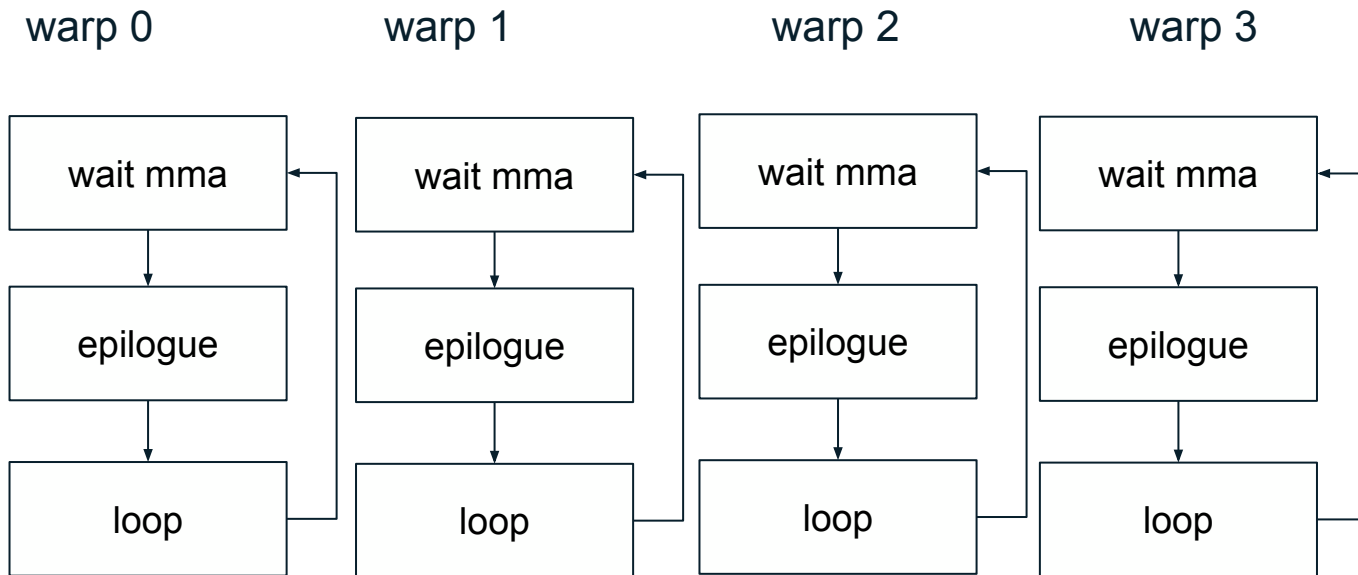
- Not warp-specialized
- Matches well with block level programming model



Why we need a new language

Blackwell

- Warp-specialization needed
- Mismatch with block level programming model
- Compiler may miss the best schedule



Introducing Gluon

Goals:

- Lower level, but still productive
- Familiar triton-like API
- Empower kernel experts
- Deliver SOL performance

Trade-offs:

- Not portable
- Harder to write



Tensor Layouts

```
@gluon.jit
def inplace_abs(x_ptr, BLOCK: gl.constexpr,
               layout: gl.constexpr):
    idxs = gl.arange(0, BLOCK, layout=layout)
    x = gl.load(x_ptr + idxs)
    gl.store(x_ptr + idxs, gl.abs(x))
```

- Defines how tensor is distributed among hardware threads
- Common types `BlockedLayout`, `SliceLayout`, etc...
- `AutoLayout`, `CoalescedLayout` allow deferring the choice
- Generalizes to any *Linear Layout*



Tensor Layouts

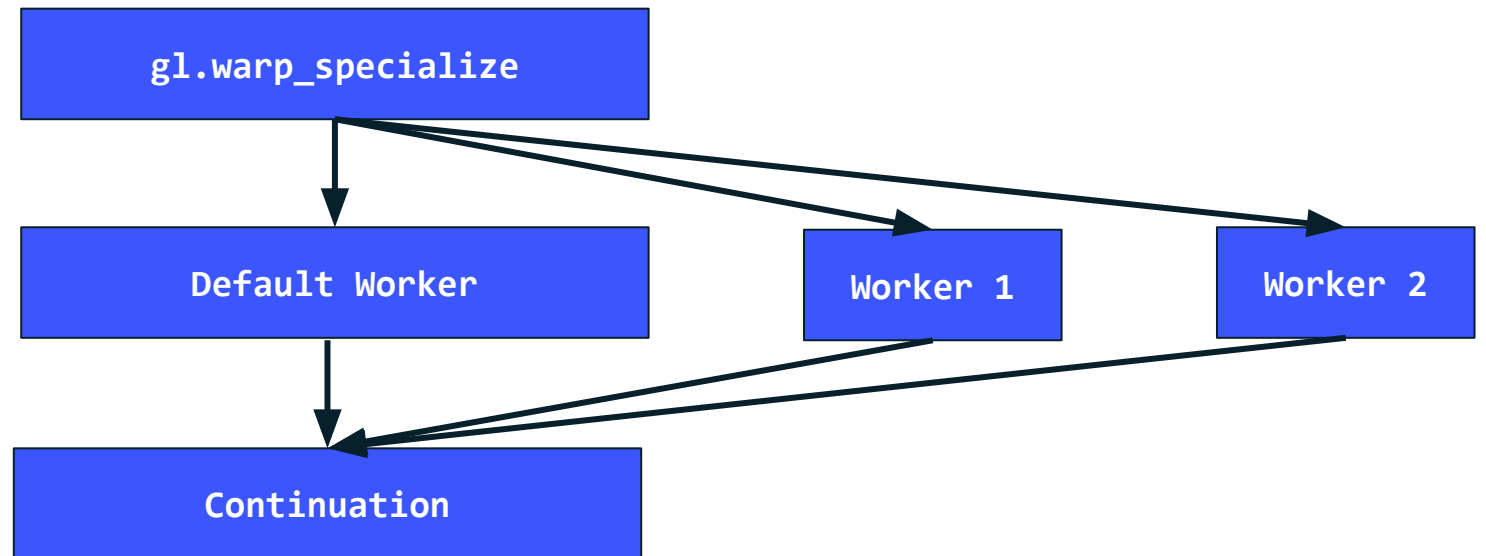
Extra control

- Indirectly control how tensor operations are lowered
- Control when to `gl.convert_layout`, or recompute
- Experiment with unusual layouts



Warp Specialization

- Fork-join model
- Default partition + Worker partitions
- Each partition runs its own sub-kernel



Shared Memory

```
smem = gl.allocate_shared_memory(  
    [3, BLOCK], layout=SwizzledSharedLayout(...))  
smem.index(0).store(x)  
y = smem.index(0).load(layout)
```

- Compile-time static allocator
- Enables communication in warp-specialized code
- Dynamic indexing enables software pipelining



Hardware intrinsics

```
from [...].nvidia.blackwell import mbarrier, tma

@gluon.jit
def load_with_tma(desc, coord, layout):
    bar = mbarrier.allocate_mbarrier()
    smem = gl.allocate_shared_memory(...)
    mbarrier.init(count=1)
    mbarrier.expect(bar, desc.nbytes_per_cta)
    tma.async_load(desc, coords, bar, smem)
    mbarrier.wait(bar, phase=0)
    return smem.load(layout)
```

- Expose hardware intrinsics e.g. tensor cores, TMA
- Requires understanding of the hardware
- Enables explicit async programming



Building Abstractions

```
@gluon.aggregate
```

```
class BarrierCounter:
```

```
    index: gl.tensor
```

```
    phase: gl.tensor
```

```
    num_barriers: gl.constexpr
```

```
@gluon.must_use_result
```

```
@gluon.jit
```

```
def increment(self):
```

```
    next_index = self.index + 1
```

```
    rollover = next_index == self.num_barriers
```

```
    index = gl.where(rollover, 0, next_index)
```

```
    phase = gl.where(rollover, self.phase ^ 1, self.phase)
```

```
    return BarrierCounter(index, phase, self.num_barriers)
```

- Aggregates combine data & code
- Aggregates are immutable



What the compiler does

Advanced lowerings

```
x = gl.load(ptrs, mask=mask, other=other)
y = gl.convert_layout(x, layout)
z = gl.sum(y, axis=-1)
```

Raising abstraction level

```
# May expand to many TMA instructions
a = tma.async_load(desc, [m, k], mbar, smem)
# May expand to many MMA instructions
tcgen05_mma(a, b, acc)
```



What the compiler does

Static memory allocation

```
# memory usage = max(smem1.nbytes, smem2.nbytes)
if cond:
    smem1 = gl.allocate_shared_memory(...)
else:
    smem2 = gl.allocate_shared_memory(...)
```



What the compiler does

Barrier insertion

```
smem.store(x)  
# Implicit bar.sync inserted  
y = smem.load(y_layout)
```

Async-proxy fence insertion

```
smem.store(result)  
# Implicit async proxy fence inserted  
tma.async_store(desc, coords, mbar, smem)
```



Comparison

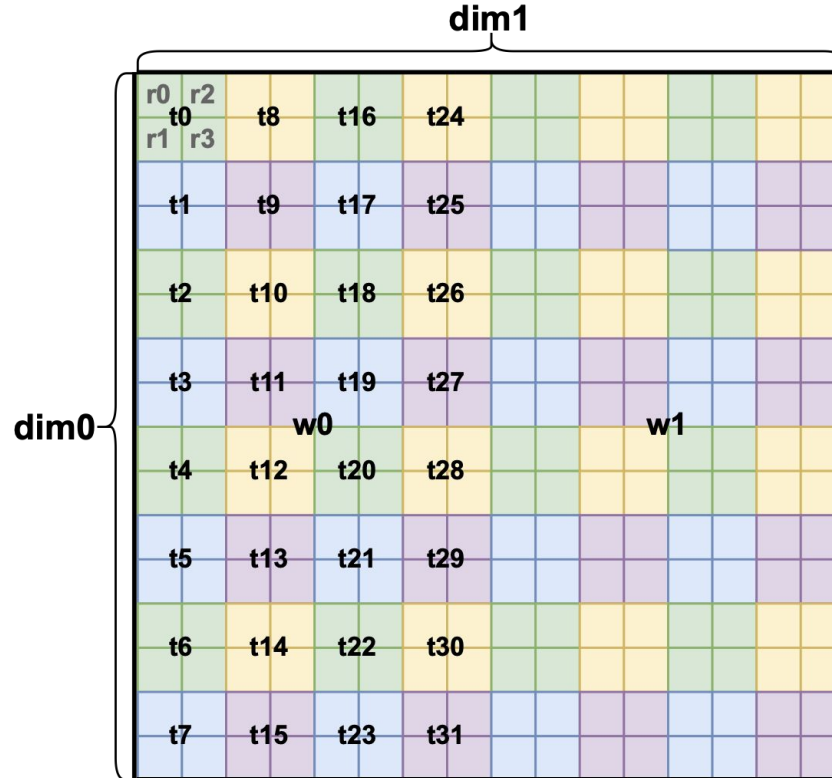
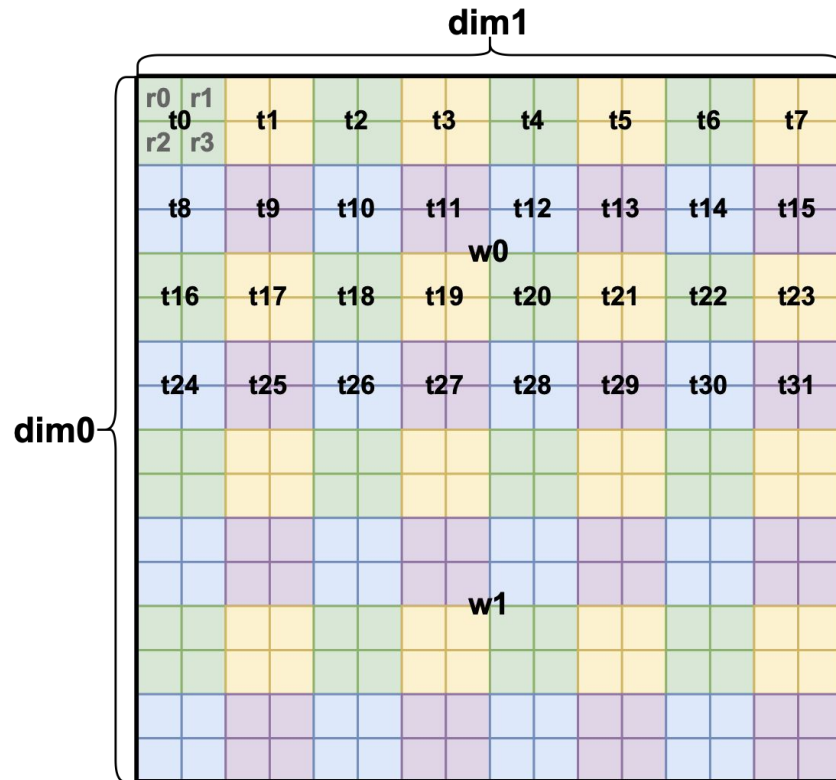
	Triton	Gluon	CuTe DSL
Abstraction	Abstract Tiles	Tiles w/ Layouts	PTX + Layout Utils
Level	Block	Warp Partitions	Thread
Performance	~80% SOL or more	SOL	SOL
Multi-Vendor	✓	✓	✗
Portable	✓	✗	✗
Layouts	Hidden	Linear Layouts	CuTe Layouts



Linear Layouts



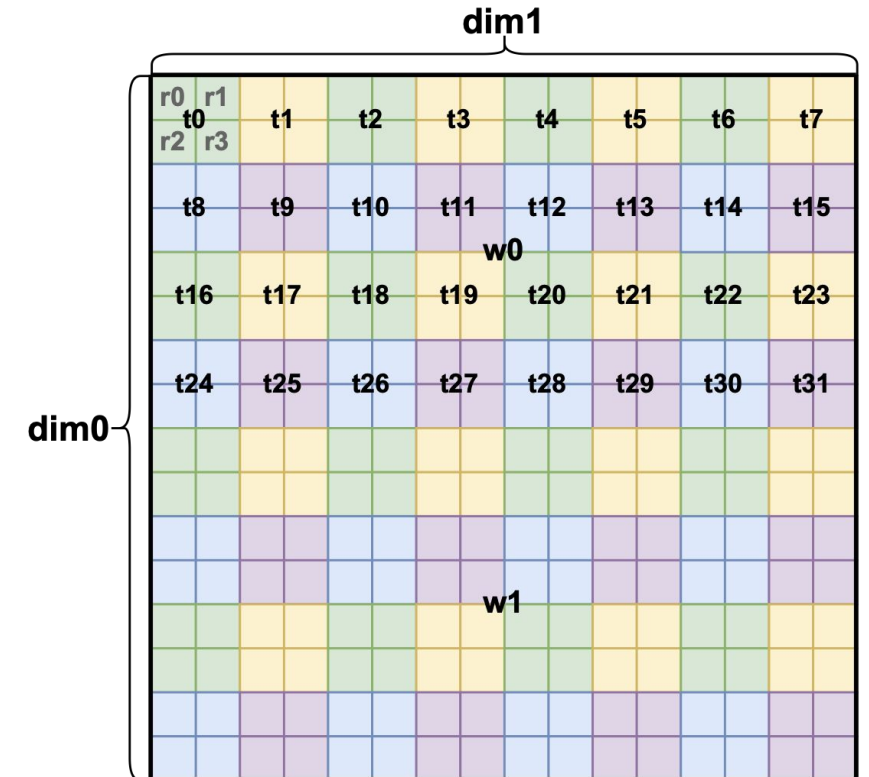
What are layouts?



What are layouts?

- Before Linear Layouts we had a number of classes of layouts to fit different uses in the program

```
ttgl.BlockedLayout(  
    size_per_thread=[2, 2],  
    threads_per_warp=[4, 8],  
    warps_per_cta=[2, 1],  
    order=[1, 0])
```



What are layouts?

Register Layouts

```
BlockedLayout([2, 2], [4, 8], [2, 1], [1, 0])  
SliceLayout(dim=1, layout)  
// mma.sync register layouts (for A, B, and C, for each dtype)  
// amd WMMA layouts
```

Shared memory layouts

```
NVMMASharedLayout(swizzle_byte_width=32, element_bitwidth=16,  
                  rank=2, transposed=False, fp4_padded=False)  
SwizzledSharedLayout(vec=1, per_phase=1, max_phase=1, order=[0])
```

Tensor memory Layouts

```
TensorMemoryLayout(block=tmem_shape, col_stride=1, two_ctas=two_ctas)  
TensorMemoryScalesLayout()
```



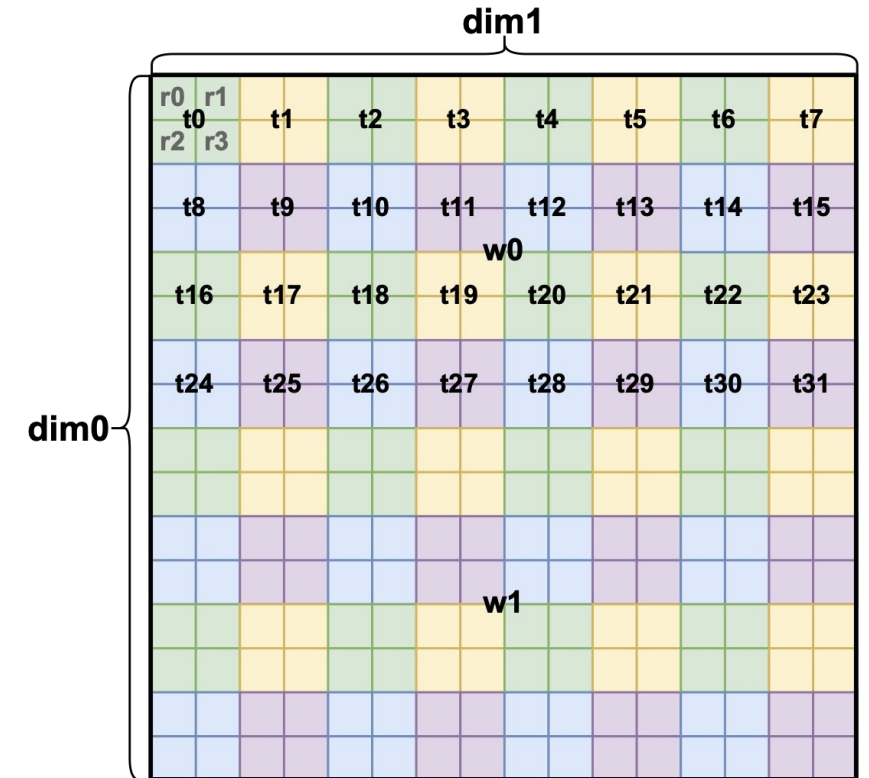
What are layouts?



What are layouts?

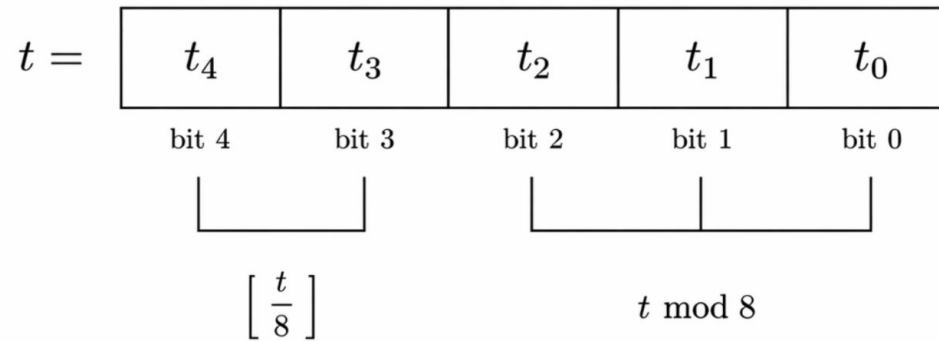
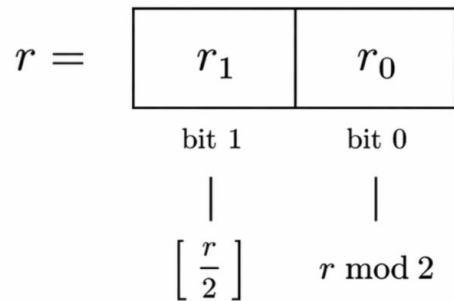
- Layouts represent how data is laid out in hardware
- A reasonable way to represent this relationship is via a function from hardware to the matrix

$$L(r, t, w) = \begin{pmatrix} \lfloor \frac{r}{2} \rfloor + 2 \lfloor \frac{t}{8} \rfloor + 8w \\ r \bmod 2 + 2(t \bmod 8) \end{pmatrix}$$

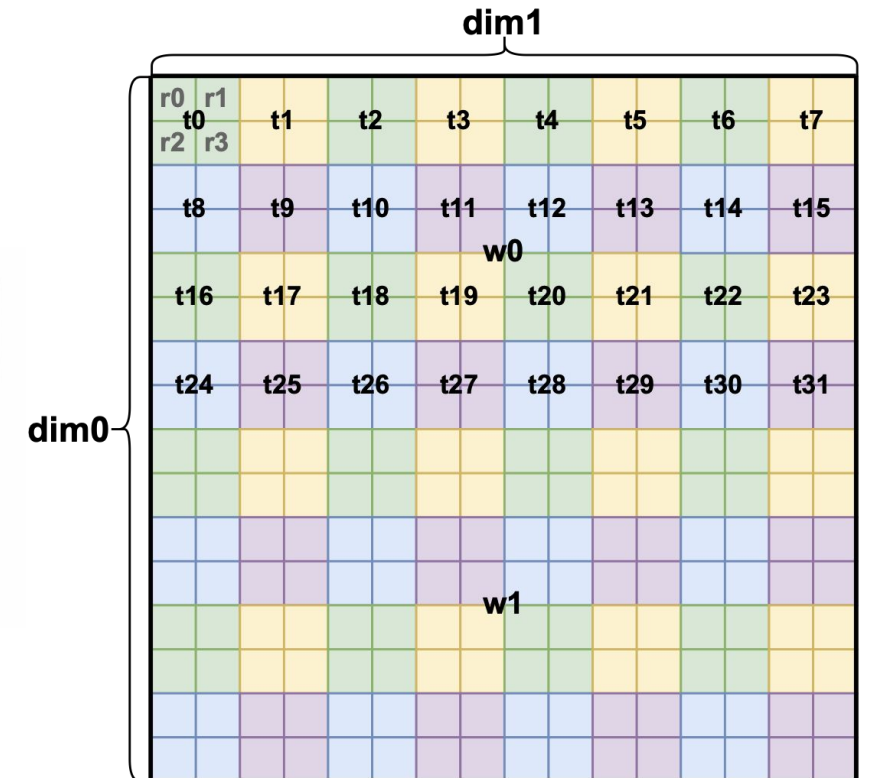


What are layouts?

$$L(r, t, w) = \begin{pmatrix} \lfloor \frac{r}{2} \rfloor + 2 \lfloor \frac{t}{8} \rfloor + 8w \\ r \bmod 2 + 2(t \bmod 8) \end{pmatrix}$$



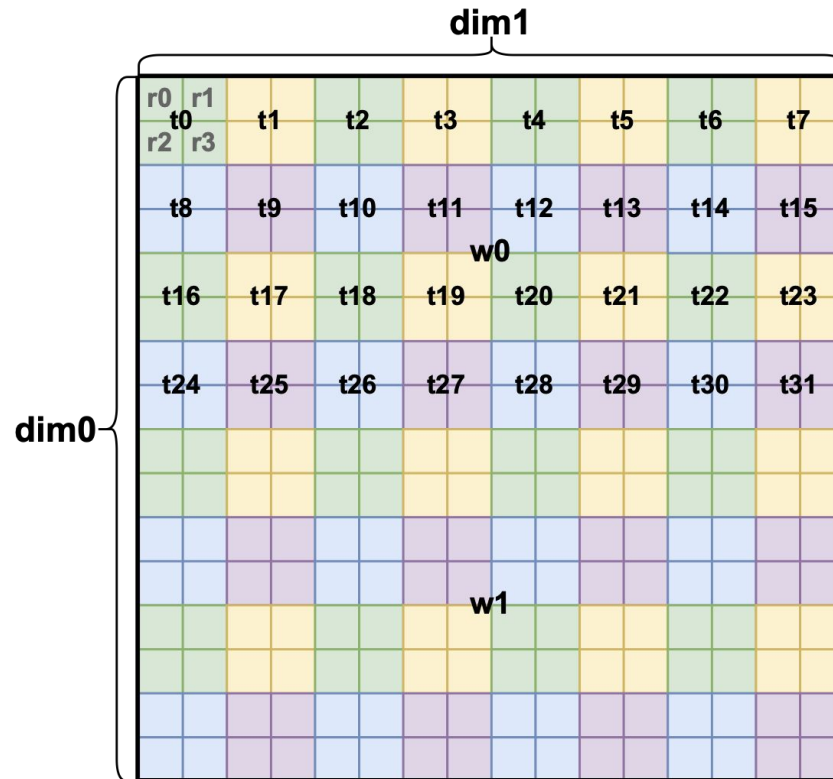
$$L(r, t, w) = \begin{pmatrix} w_0 \ t_4 \ t_3 \ r_1 \\ t_2 \ t_1 \ t_0 \ r_0 \end{pmatrix}$$



What are layouts?

$$L(r, t, w) = \begin{pmatrix} w_0 & t_4 & t_3 & r_1 \\ t_2 & t_1 & t_0 & r_0 \end{pmatrix}$$

$$L(r, t, w) = \left(\begin{array}{cc|cccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right) \begin{pmatrix} r_0 \\ r_1 \\ t_0 \\ t_1 \\ t_2 \\ t_3 \\ t_4 \\ w_0 \end{pmatrix} = \begin{pmatrix} r_0 \\ t_0 \\ t_1 \\ t_2 \\ r_1 \\ t_3 \\ t_4 \\ w_0 \end{pmatrix} = \begin{pmatrix} \overbrace{t_2 \ t_1 \ t_0 \ r_0}^{\text{dim 1}} \\ \underbrace{w_0 \ t_4 \ t_3 \ r_1}_{\text{dim 0}} \end{pmatrix}$$



```
ttgl.DistributedLinearLayout(
    reg_bases = [[0, 1], [1, 0]],
    lane_bases = [[0, 2], [0, 4], [0, 8], [2, 0], [4, 0]],
    warp_bases = [[8, 0]],
    shape = [16, 16])
```



Examples of Linear Layouts: mma.sync

```
ttgl.NVMMA DistributedLayout(
    version=[2, 0],
    warps_per_cta=[4, 1],
    instr_shape=[16, 8])

ttgl.DistributedLinearLayout(
    reg_bases = [[0, 1], [8, 0], [0, 8]],
    lane_bases = [[0, 2], [0, 4],
                  [1, 0], [2, 0], [4, 0]],
    warp_bases = [[16, 0], [32, 0]],
    shape=[64, 16])
```

R\C	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0:{a0,a1}	T1:{a0,a1}	T2:{a0,a1}	T3:{a0,a1}					T0:{a4,a5}	T1:{a4,a5}	T2:{a4,a5}	T3:{a4,a5}				
1	T4:{a0,a1}	T5:{a0,a1}	T6:{a0,a1}	T7:{a0,a1}					T4:{a4,a5}	T5:{a4,a5}	T6:{a4,a5}	T7:{a4,a5}				
2	→								→							
..																
7	T28:{a0,a1}	T29:{a0,a1}	T30:{a0,a1}	T31:{a0,a1}					T28:{a4,a5}	T29:{a4,a5}	T30:{a4,a5}	T31:{a4,a5}				
8	T0:{a2,a3}	T1:{a2,a3}	T2:{a2,a3}	T3:{a2,a3}					T0:{a6,a7}	T1:{a6,a7}	T2:{a6,a7}	T3:{a6,a7}				
9	T4:{a2,a3}	T5:{a2,a3}	T6:{a2,a3}	T7:{a2,a3}					T4:{a6,a7}	T5:{a6,a7}	T6:{a6,a7}	T7:{a6,a7}				
10	→								→							
..																
15	T28:{a2,a3}	T29:{a2,a3}	T30:{a2,a3}	T31:{a2,a3}					T28:{a6,a7}	T29:{a6,a7}	T30:{a6,a7}	T31:{a6,a7}				

%laneid:{fragments}

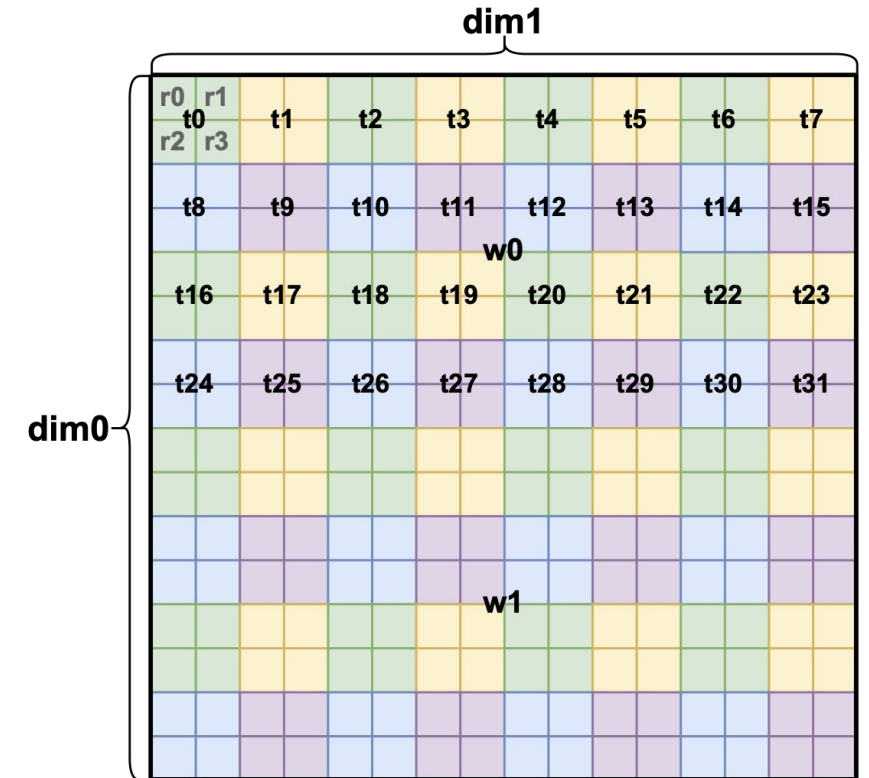


Examples of Linear Layouts: Reduction / Broadcasting

```
linear =  
ttgl.DistributedLinearLayout(  
    reg_bases = [[0, 1], [1, 0]],  
    lane_bases = [[0, 2], [0, 4], [0, 8], [2, 0], [4, 0]],  
    warp_bases = [[8, 0]],  
    shape = [16, 16])
```

```
gl.static_assert(x.layout == linear)  
y = gl.sum(x, dim=1)  
gl.static_assert(y.layout == SliceLayout(dim=1, linear))
```

```
slice =  
ttgl.DistributedLinearLayout(  
    reg_bases = [[1]],  
    lane_bases = [[0], [0], [0], [2], [4]],  
    warp_bases = [[8]],  
    shape = [16])
```



Linear Layouts: Definition

Definition:

A **Linear Layout** is a linear map over $\mathbb{F}_2$.

- Recall that $\mathbb{F}_2 = \{0, 1\}$ with the XOR as the addition and the AND as the multiplication
- In simpler words: a Linear Layout is a binary matrix that we multiply by a vector using XORs and ANDs in place of addition and multiplication



Linear Layouts: Swizzling

$$L(o) = \begin{pmatrix} (o \bmod 8) \wedge \lfloor \frac{o}{8} \rfloor \\ \lfloor \frac{o}{8} \rfloor \end{pmatrix}$$

$$L(o) = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} o_1 \\ o_2 \\ o_3 \\ o_4 \\ o_5 \\ o_6 \end{pmatrix} = \begin{pmatrix} o_1 \wedge o_4 \\ o_2 \wedge o_5 \\ o_3 \wedge o_6 \\ o_4 \\ o_5 \\ o_6 \end{pmatrix}$$

Diagram illustrating a 2D array layout (M/N dim vs K dim (contiguous)) with a swizzling pattern. The array is 8x8, and the elements are numbered 0 to 63. The M/N dim is the row index (0 to 7), and the K dim (contiguous) is the column index (0 to 7). The elements are colored in a repeating 2x2 pattern of 4 colors (blue, green, yellow, red) to represent the swizzling.

	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	9	8	11	10	13	12	15	14
2	18	19	16	17	22	23	20	21
3	27	26	25	24	31	30	29	28
4	36	37	38	39	32	33	34	35
5	45	44	47	46	41	40	43	42
6	54	55	52	53	50	51	48	49
7	63	62	61	60	59	58	57	56



What have the Linear Layouts ever done for us?

- **Well-defined:** They give a unified framework on top of which to build Gluon
 - This has fixed the source of most of previous bugs in Triton and has enabled Gluon
- **Simplicity:** It's just good ol' linear algebra
- **Completeness:** Translates hardware problems into linear algebra problems:
 - If my data is distributed with layout A in registers and I want to put it in shmem with layout B, which indexing should I generate for the element in a register/lane/warp?

$$\text{offset} = B^{-1}A(r, l, w)$$



What have the Linear Layouts ever done for us?

- **Generic lowerings:** We can write optimisations generically in the lowerings as we know the full algebraic structure of the inputs
 - We can decide whether we can use ``stmatrix`` or ``stmatrix.trans`` or ``st.shared``, and which vectorisation to use, regardless of the specific register and shmem layouts
- **Optimal layout conversions:**
 - We derive the optimal swizzling (maximise vectorisation and minimise bank conflicts using arbitrary instructions (`ld.shared`, `ldmatrix`, etc)) for any pair of linear layouts
 - It automatically generates optimal swizzling for matrix transposes and other funky patterns
 - If the layouts can be lowered via shuffles, we derive the optimal shufflings sequence
 - It automatically generates the FA3 hand-tuned shuffling sequence



If you want to know more

**Linear Layouts: Robust Code Generation of
Efficient Tensor Computation Using $\mathbb{F}_2$**

<https://arxiv.org/pdf/2505.23819>

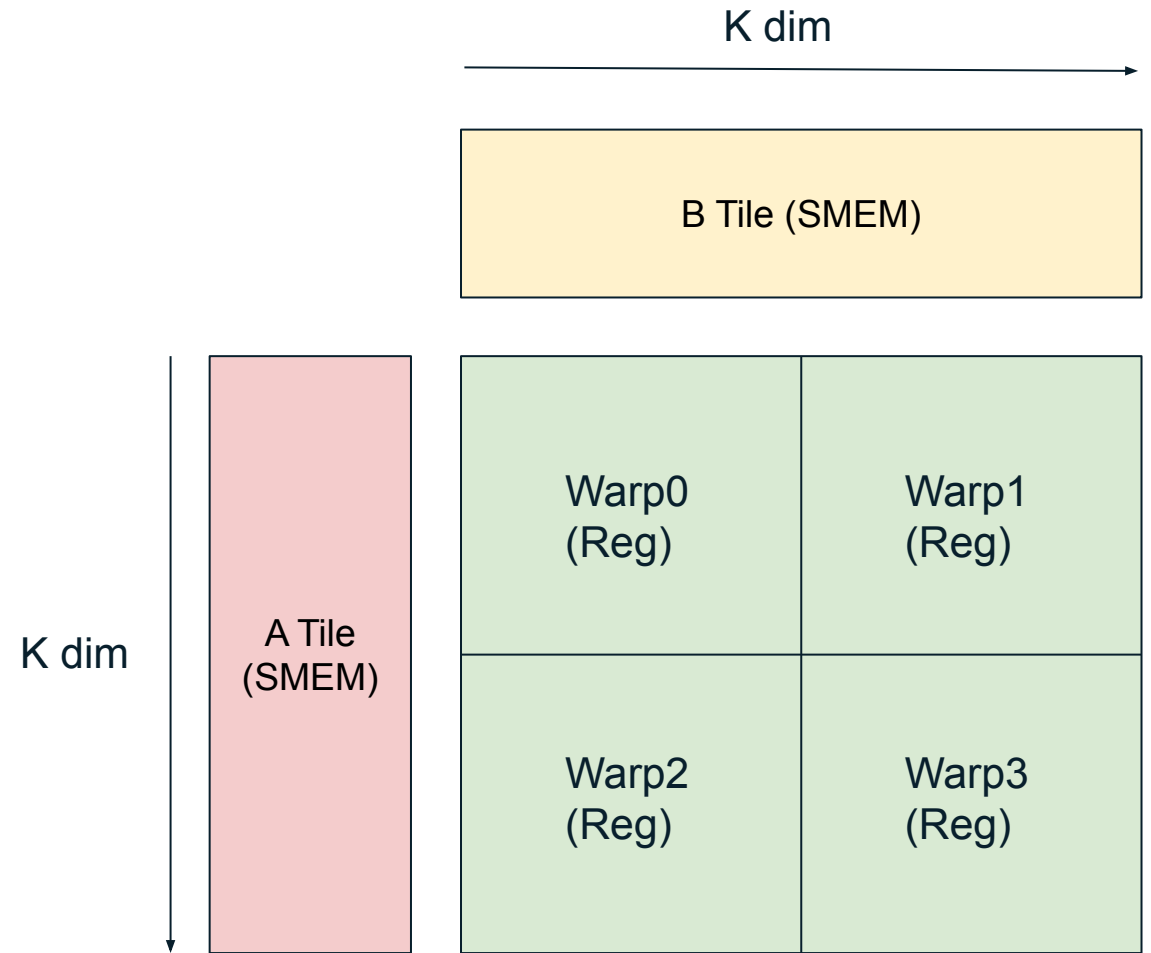


Optimized Blackwell Matmul



Review: Matmul on GPUs

- $A \times B = C$
- Parallelism
 - Each CTA taking rows and columns from A and B
- Data pipeline
 - Global memory
 - Shared memory
 - Registers



Matmul with tcgen05

- **Tensor memory:** Used to stage the **operands and accumulator** for tcgen05.mma

Shared Memory $\leftarrow$ `st.shared / cp.async`

$\downarrow$
`tcgen05.mma`

$\downarrow$
Tensor Memory $\rightarrow$ C tile

$\downarrow$
Registers $\leftarrow$ `tcgen05.ld`

$\downarrow$
Shared Memory $\leftarrow$ `st.shared`

$\downarrow$
Global Memory $\leftarrow$ `cp.async`

```
tmem_layout = TensorMemoryLayout(  
    [BLOCK_SIZE_M, BLOCK_SIZE_N],  
    col_stride=1,  
    cga_layout=CGA_LAYOUT,  
    two_ctas=TWO_CTAS,  
)
```

```
smem_layout = gl.NVMMASharedLayout.get_default_for(  
    block_shape,  
    dtype,  
    cga_layout  
)
```



Use tcgen05 in Gluon

- Allocate tensor memory with a proper layout and tile size
- Copy data from global memory to shared memory
- Call `tcgen05_mma`
- Store data to global memory
 - tmem to reg
 - reg to smem
 - smem to gmem

```
tma_bar = mbarrier.allocate_mbarrier(...)
mma_bar = mbarrier.allocate_mbarrier(...)

mbarrier.init(tma_bar, count=1)
mbarrier.init(mma_bar, count=1)

a_smem = gl.allocate_shared_memory(...)
b_smem = gl.allocate_shared_memory(...)
c_smem = gl.allocate_shared_memory(...)
tmem = allocate_tensor_memory(...)

bytes = a_smem.nbytes_per_cta +
        b_smem.nbytes_per_cta
mbarrier.expect(tma_bar, bytes)

tma.async_load(a_smem, ...)
tma.async_load(b_smem, ...)
mbarrier.wait(tma_bar, 0)

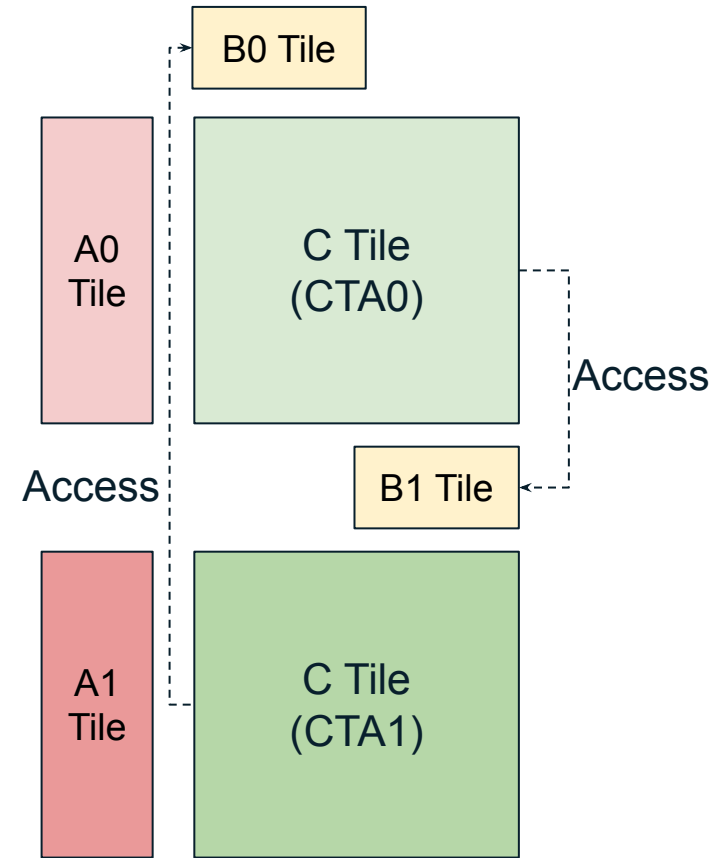
tcgen05_mma(a_smem, b_smem, tmem, mma_bar)
mbarrier.wait(mma_bar, 0)

c_smem.store(tmem.load())
tma.async_store(..., c_smem)
```



Matmul Optimizations: 2-CTA

- `tcgen05.mma` in 2-CTA mode makes CTA0 and CTA1 share their B tiles to compute their output tile
- Enables one larger MMA with reduced shared memory consumption (no need to replicate the B tile)



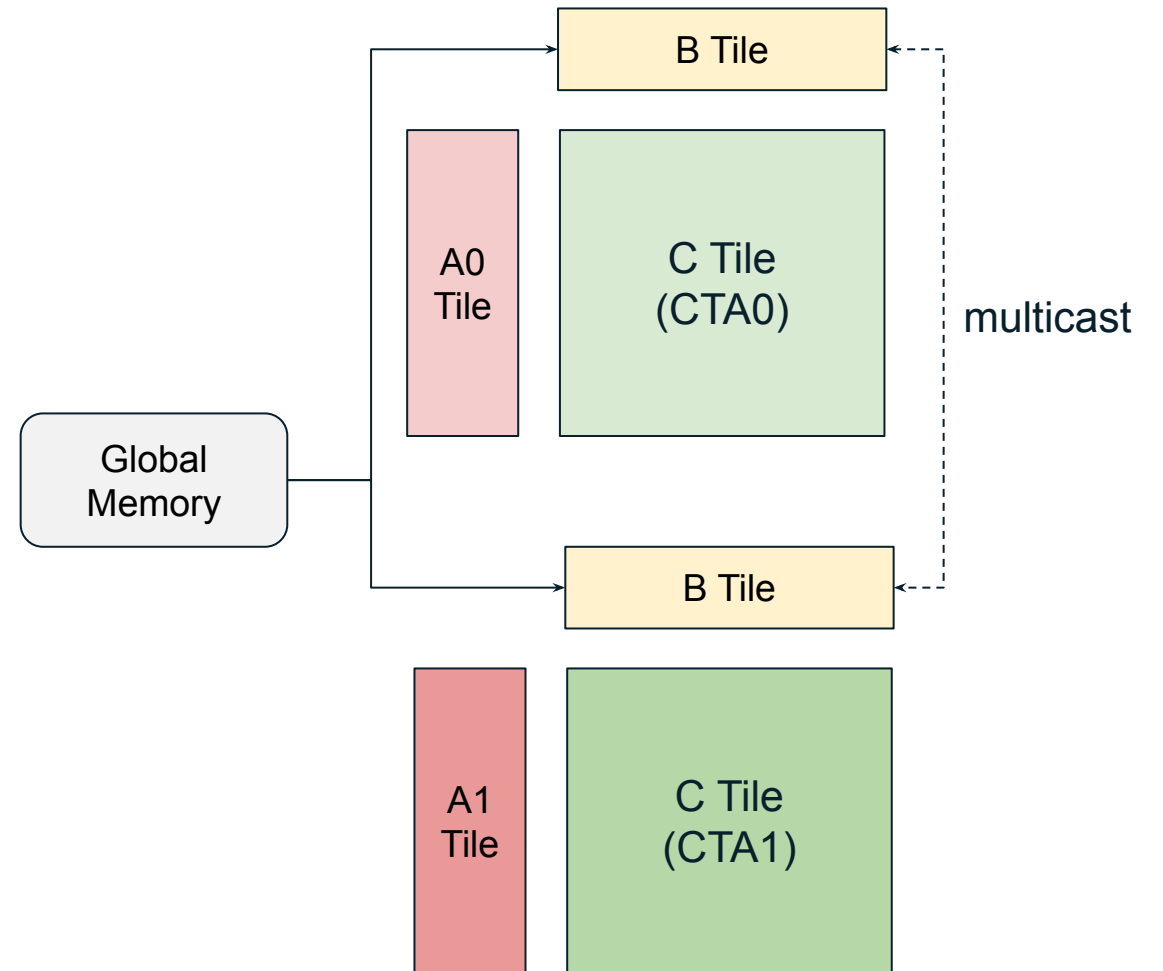
Use 2-CTA in Gluon

- Set the `two_ctas` flag in relevant operations
 - `mbarrier.allocate_mbarrier(..., two_ctas=True)`
 - `TensorMemoryLayout(..., two_ctas=True)`
- Use the correct `cga_layout` for loading operand tiles
 - `tcgen05_mma` will figure out the two-cta mode based on the layout
 - If you get them wrong your program will not compile



Matmul Optimizations: TMA Multicast

- Ampere
 - CTA0 and CTA1 load the same B tile from global memory
- Hopper/Blackwell
 - TMA multicast issued by one CTA (e.g., CTA0) replicates the tile into multiple CTAs' shared memory (CTA0/CTA1)
 - Reduces L2 traffic



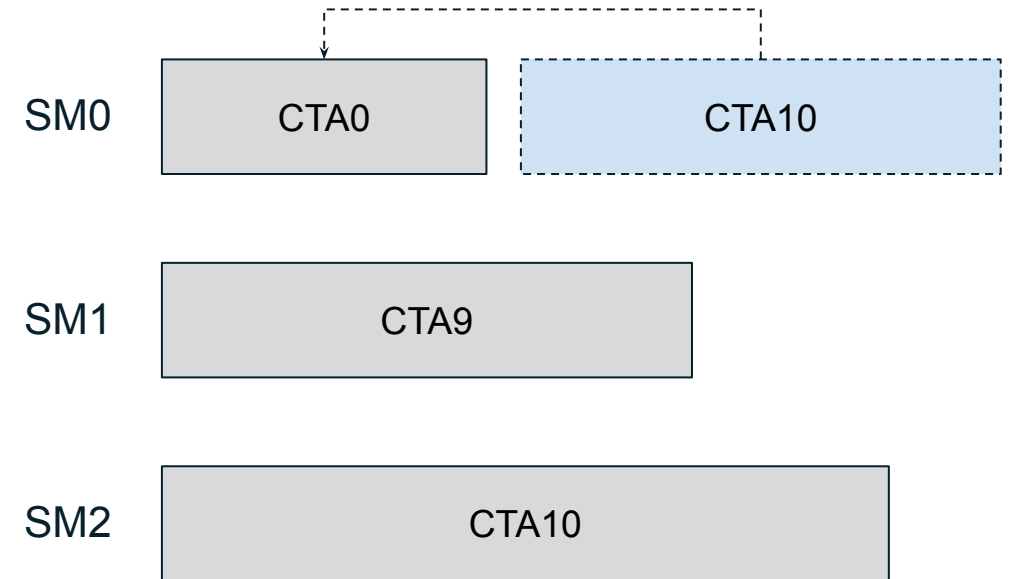
Use TMA Multicast in Gluon

- Specify CGA layout
 - A linear layout that specifies how to split the tensor on M/N dimensions
 - lhs: `[(1, 0), (2, 0)]`
 - CTAs evenly split the M dimension
 - rhs: `[(0, 1), (0, 0)]`
 - CTA0 and CTA1 evenly split the N dimension (2CTA mode)
 - The data is repeated on CTA2 and CTA3 (Multicast)
 - res: `[(1, 0), (2, 0)]`
- Set the multicast flag in the relevant operations
 - `tcgen05_mma_barrier_count(..., multicast=True)`
 - `tma.async_load(..., multicast=True)`
 - `tcgen05_mma(..., multicast=True)`



Matmul Optimizations: Cluster Launch Control

- Static persistent kernels
 - #CTAs = #SMs
 - Sometimes workload can be imbalanced
 - Idle SMs cannot be scheduled
- Dynamic scheduling
 - Schedule new workload on existing CTAs



Use Cluster Launch Control (CLC) in Gluon

- Initialize a warp group for CLC workload scheduling
- Wait for other warp groups (e.g., load/mma/store) arrive at the barrier
- Cancel scheduling of the next work tile
- Notify other warp groups that the tile ID is ready

```
@gluon.aggregate
class ClcTileScheduler:
    @gluon.jit
    def initialize(M, N, BLOCK_M, BLOCK_N):
        ...

    @gluon.jit
    def try_cancel(self) -> None:
        clc.try_cancel(self.clc_result_buf, self.barrier)
        mbarrier.expect(self.barrier, 16)

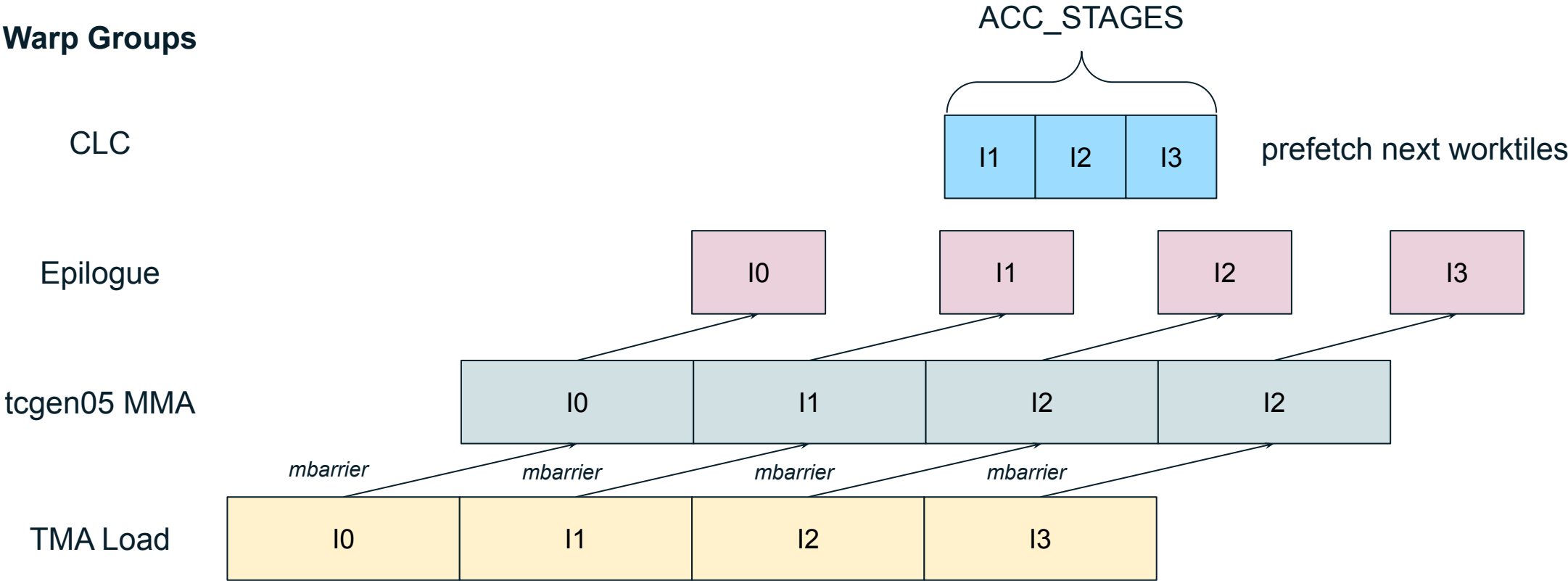
    @gluon.jit
    def advance(self):
        mbarrier.wait(self.barrier, self.phase)
        clc_res = clc.load_result(self.clc_result_buf)
        has_work = clc_res.is_canceled()
        next_tile_id = clc_res.program_id(0)
        return ClcTileScheduler(...)
```



SOL Matmul Diagram

```
gl.warp_specialize
```

Warp Groups



More Gluon

Tutorials

<https://triton-lang.org/main/getting-started/tutorials/gluon/index.html>

Examples:

<https://triton-lang.org/main/getting-started/examples/gluon/index.html>



Profilers



Proton

- Designed for flexible, low-overhead, and iterative profiling and optimization of GPU kernels, especially Triton
- A unique `session_id` is returned from the `start` invocation
- Proton profiler API
 - `proton.start/proton.finalize`
- Proton instrumentation API
 - `proton.start(backend="instrumentation")`
- Proton PC sampling API
 - `proton.start(backend="cupti", mode="pcsampling")`



Proton Profiler APIs

- Choose data to collect
- Choose regions of kernels to profile
 - cache clear kernels are not included in the final profile
 - **activate/deactivate**
- Save profile to the disk
 - **finalize**

```
def run_profile(shape, a, b, c_torch, run_gluon):  
    import triton.profiler as proton  
  
    M, N, K = shape  
  
    proton.start("matmul")  
    proton.deactivate()  
  
    def bench_fn(label, reps, fn):  
        proton.deactivate()  
        for _ in range(5):  
            fn() # warmup  
            clear_cache(l2_cache)  
        try:  
            for _ in range(reps):  
                proton.deactivate()  
                clear_cache(l2_cache)  
                proton.activate()  
                fn()  
        finally:  
            proton.deactivate()
```



Coarse-Grained Profiling

- TFLOPS benchmarked on B200
- Gluon's perf is on par with cuBLAS
 - without exhaustive autotuning
 - both have 2cta enabled

```
[M=4096, N=8192, K=4096]
├─ 1643.246 16.728 gluon
│ └─ nan 16.728 _matmul_kernel
└─ 1627.883 16.886 torch
   └─ nan 16.886 nvjet_sm100_hsh_128x256_64x6_2x1_2cta_v_bz_NNT
[M=4096, N=4096, K=4096]
├─ 1548.387 8.876 gluon
│ └─ nan 8.876 _matmul_kernel
└─ 1540.391 8.922 torch
   └─ nan 8.922 nvjet_sm100_hsh_128x256_64x6_2x2f_2cta_h_bz_NNT
[M=4096, N=8192, K=2048]
├─ 1655.533 8.302 gluon
│ └─ nan 8.302 _matmul_kernel
└─ 1665.878 8.250 torch
   └─ nan 8.250 nvjet_sm100_hsh_128x256_64x6_2x1_2cta_v_bz_NNT
```



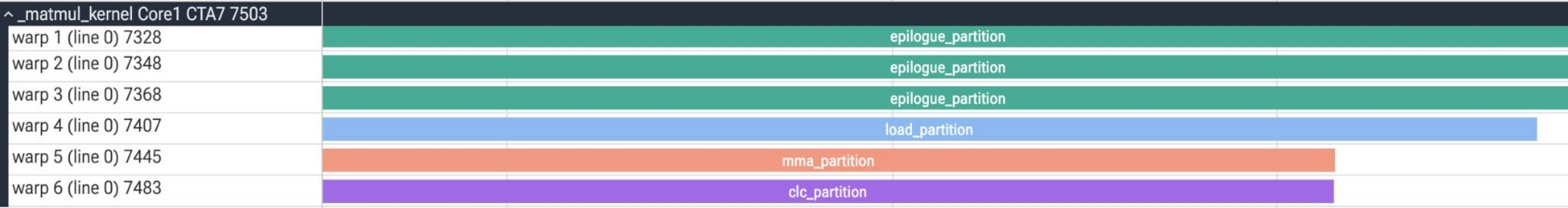
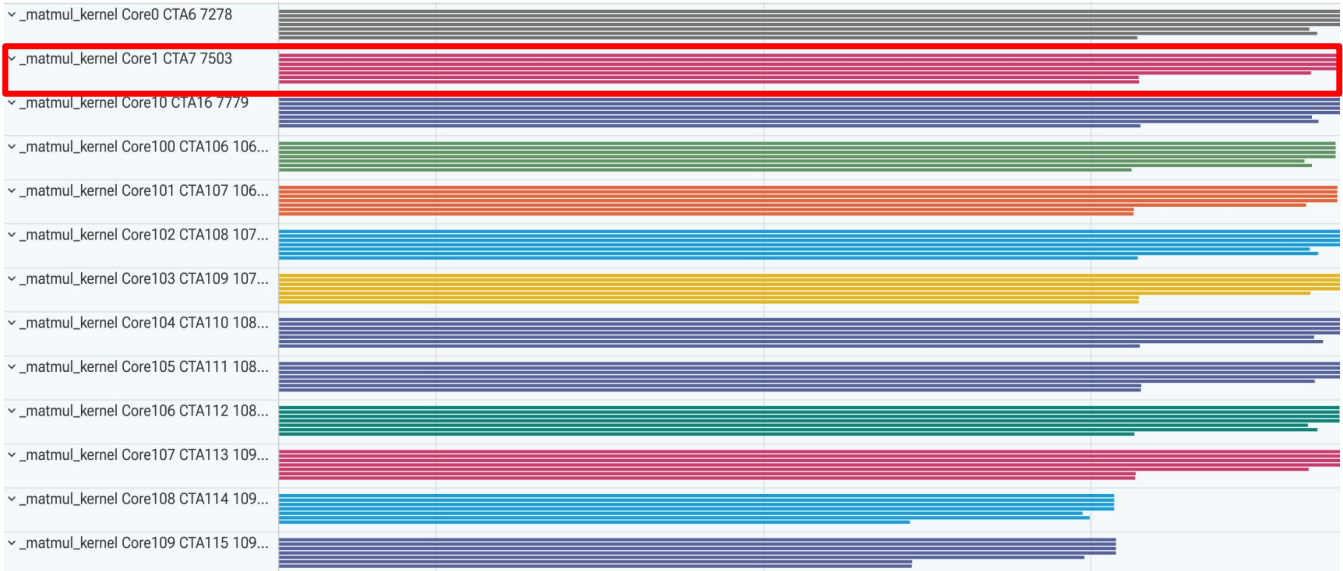
Proton Instrumentation APIs

- Host side
 - Choose the `instrumentation` backend
 - Configure profiling options using the `mode` argument
 - Finalize profiling after each kernel
- Device side
 - Annotate regions using APIs or a context manager

```
def matmul_load_partition(p):  
    ...  
    pl.enter_scope("load_partition")  
    while scheduler.has_work:  
        ...  
    pl.exit_scope("load_partition")  
  
def matmul_mma_partition(p):  
    ...  
    pl.enter_scope("mma_partition")  
    while scheduler.has_work:  
        ...  
    pl.exit_scope("mma_partition")
```



Fine-grained Profiling



Sanitizers



Sanitizers

Concurrency Sanitizer (consan)

- Detects races within a gluon program, e.g.
 - Using TMA load before waiting on the mbarrier
 - Overwriting MMA inputs while MMA is using them

Invalid Instruction Sanitizer (iisan)

- Adds alignment checks on TMA load/store operations

```
triton.knobs.compilation.instrumentation_mode = "consan,iisan"
```



Sanitizers

Floating Point Sanitizer (fpsan)

- Replaces floating point arithmetic with special integer operations
 - Preserve mathematical identities like associativity
 - $(a + b) + c == a + (b + c)$ [under fpsan]
- Enables testing optimizations that change rounding, without relying on a tolerance like `assert_close`

Global Memory Sanitizer (gsan)

- Runtime check for race conditions between CTAs/gluon programs
- Supports nvlink comms and cross-kernel synchronization
- Note: must allocate tensors with `gsan.create_mem_pool()`

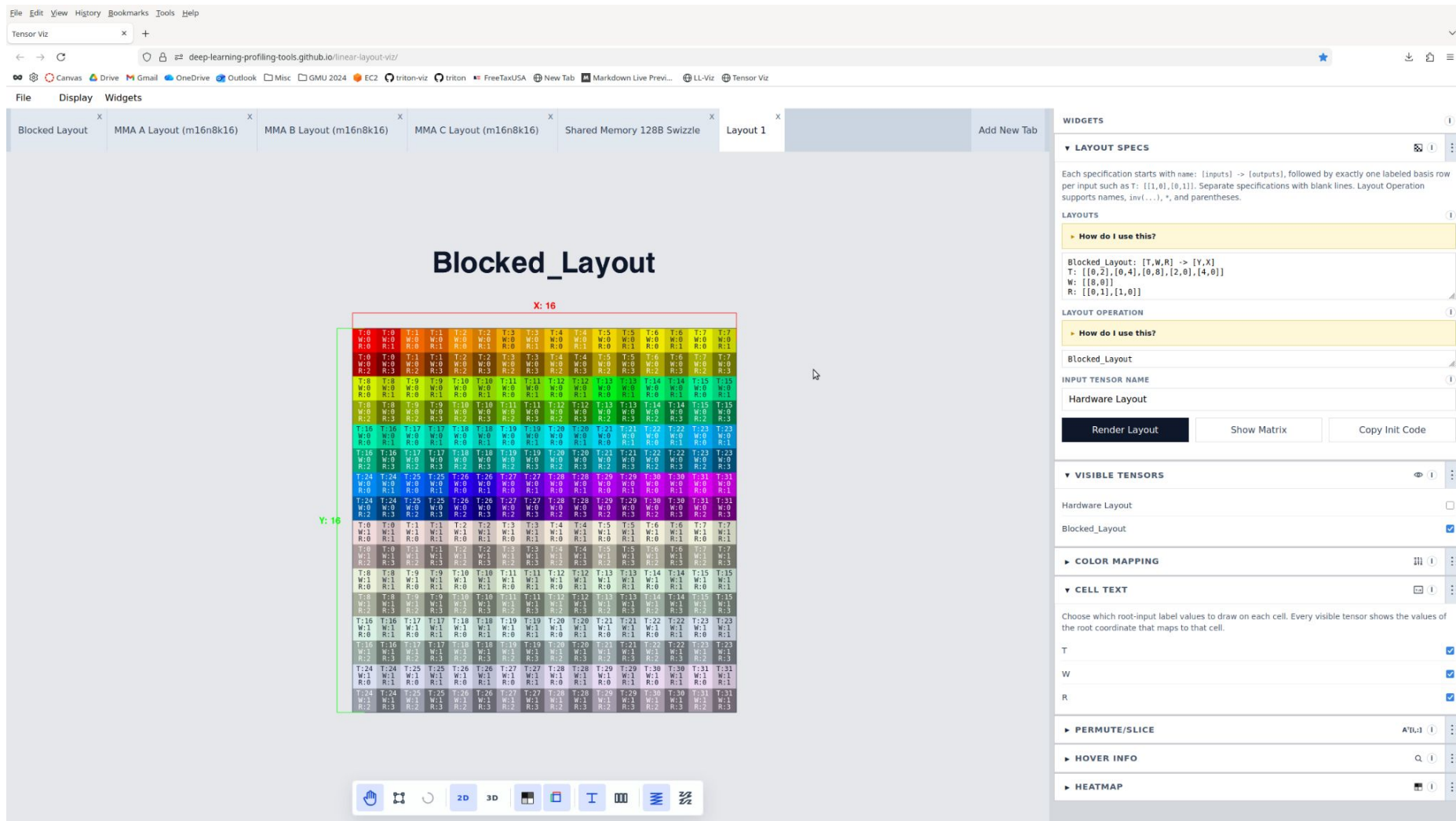
```
triton.knobs.compilation.instrumentation_mode = "fpsan,gsan"
```



Linear Layout Visualizer



Interactive Visualization of Linear Layouts



Developed by Thaihoa Nguyen @sardine



Purposes

- Learn existing layouts (e.g., MMA)
- Debug custom layouts
 - New hardware
 - Reshape / split / join triton operators
- Debug linear layout operators
 - Compose / invert



Access

- Website
 - <https://deep-learning-profiling-tools.github.io/linear-layout-viz/>
- Code
 - <https://github.com/Deep-Learning-Profiling-Tools/linear-layout-viz>
- Manual
 - <https://github.com/Deep-Learning-Profiling-Tools/linear-layout-viz/blob/main/MANUAL.md>



Main Functions

- Selection
 - Load preset layouts
- Tabs
 - Switch between different layouts
- Operators
 - Apply linear layout operators and view results
- Slicing
 - Check locations of each register/thread/warp



Takeaway

- Gluon exposes hardware features
 - First-class primitives for hardware-specific capabilities
 - Explicit layout control for fine-grained optimization
- Gluon eases programming with its design
 - Triton-like programming model and abstractions
 - Automatic memory allocation and synchronization (barriers)
- Fine-grained profiling for Gluon is accurate due to minimal compiler transformations





Thank you

Peter Bell, Mario Lezcano, Keren Zhou
OpenAI

Backup Slide: Proton Common Errors

- Unexpected high overhead
- High shared memory usage
 - `triton.runtime.errors.OutOfResources: out of resource: shared memory, Required: 295144, Hardware limit: 232448`
- Solution
 - Change `buffer_type`, `buffer_size`, and `sampling_strategy` accordingly in `mode.Default`

